과제 #3

M1522.006700 확장형 고성능 컴퓨팅 (001)

2021-15531 컴퓨터공학부 김인호
October 26, 2025

1 Matrix Multiplication using OpenMP

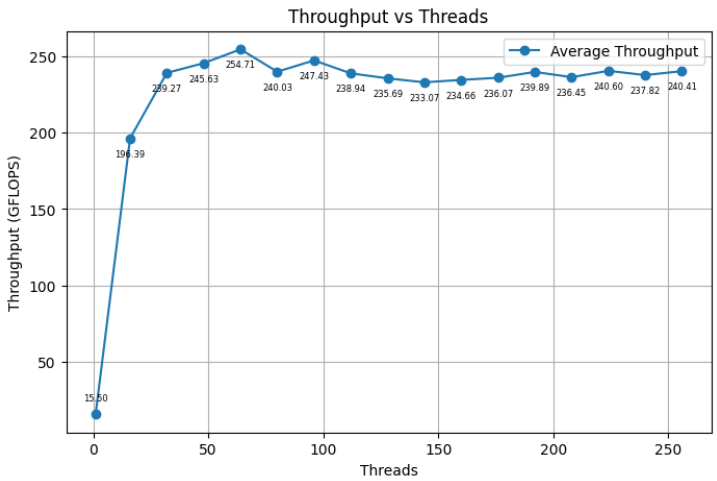
- **병렬화 방식**
  행렬곱을 구하는 반복문의 가장 바깥 루프 윗줄에

  ```
  #pragma omp parallel for num_threads(num_threads)
  ```

  을 넣어 OpenMP에서 `num_threads`개만큼의 스레드를 생성하고 자동으로 for문을 각 스레드가 나누어 계산할 수 있도록 하였다. 가장 바깥의 반복문의 `its`를 나눈다고 하면, 여러 스레드가 공유하는 메모리는 행렬 B뿐이고 쓰기가 진행될 행렬 C는 공유하지 않으므로, `critical`이나 `barrier` 같은 키워드를 통한 영역 구분이 필요 없다. 또한, 과제 2번에서 한 것처럼 `BLOCK_SIZE`를 128로 정해 각 행렬을 (128, 128)로 조각내어 연산할 수 있도록 함으로써, 캐시를 효율적으로 활용할 수 있도록 하였다.

- **OpenMP의 스레드 생성 방식** OpenMP에서는 명시적으로 스레드를 생성하는 함수를 호출하지 않고, 사전에 스레드의 개수만 지정해 주면 `#pragma omp parallel`과 같은 키워드가 붙은 코드에서 자동으로 스레드를 생성하고 각 스레드에 작업을 할당한다. 컴파일러는 이러한 지시문을 해석하여, 지시에 맞게 실제로 스레드를 지시 및 관리하는 코드들을 삽입하는 식으로 번역 과정을 수행한다. 프로그램을 실행할 때에는 런타임 시스템이 fork와 join 등을 OS 수준에서 호출하여 스레드를 생성하고 관리하는 역할을 실제로 수행한다.

- **스레드 수에 따른 행렬곱 성능** 과제 2와 같은 방법으로 스레드를 1개 사용하였을 때와, 스레드의 개수를 16씩 늘려가며 스레드를 16개부터 256개까지 사용하였을 때의 성능을 측정하였다. 1회 측정 시 각 스레드 수에 대해 $M = N = K = 4096$으로 두고 5번 반복 측정하였으며, 이러한 측정을 또 4번 반복하여 각 스레드 개수 별 성능(GFLOPS)의 평균을 아래와 같은 그래프와 표로 나타내었다.



Threads	성능 (GFLOPS)	Threads	성능 (GFLOPS)
1	15.503946	144	233.069864
16	196.389475	160	234.660170
32	239.270952	176	236.072343
48	245.630547	192	239.887478
64	254.708554	208	236.453157
80	240.028683	224	240.601272
96	247.432433	240	237.816317
112	238.937493	256	240.413497
128	235.694766		

성능 측정 결과, 스레드의 개수가 64가 될 때까지는 성능이 크게 상승하다가 그 뒤로는 240 GFLOPS 내외에서 유지되는 모습을 확인할 수 있었다. 이는 스레드 수가 많아질수록 병렬화로 인한 성능 상승의 효과도 있지만, 메모리 대역폭으로 인한 병목이나 스레드 관리의 오버헤드 등 성능 하락 요인 또한 존재하기 때문으로 여겨진다.

- **OpenMP의 loop scheduling**
  OpenMP에서는 스레드에 반복문의 iteration을 여러 방식으로 나눌 수 있는데, 그 중 `static`은 스레드 번호 순으로 돌아가며 항상 정해진 크기(`chunk_size`)만큼의 청크를 받는 방식이다. 항상 균일하게 나눈 단순한 방식이므로 오버헤드가 적다.

  `dynamic`은 해당 프로그램에 배정된 스레드가 요청할 때 정해진 크기의 청크를 배분해주는 방식으로, 나누어줄 청크가 없을 때까지 일을 막 마친 스레드가 남은 청크를 요청하고 가져간다. iteration마다 필요한 작업이 균등하지 않을 때 좋지만, 오버헤드가 크다.

  `guided`는 `dynamic`과 방식은 같지만, 초반에는 `chunk_size`보다 더 큰 크기의 청크를 할당하다가 할당하는 청크의 크기를 정해둔 크기까지 점점 줄이는 방식이다. `dynamic`보다 청크를 나누어주는 횟수를 줄여 동적인 할당임에도 오버헤드를 줄일 수 있다.

  행렬 크기와 스레드 수, 그리고 스케줄링 방식에 따른 성능을 테스트한 결과는 아래와 같았다.

실험 환경	`static`	`dynamic`	`guided`
$N = M = K = 1024$, 128 threads	80.720312	51.084957	51.495321
$N = M = K = 4096$, 32 threads	334.864415	336.565073	338.476523
$(N, M, K) = (6329, 3217, 5021)$, 32 threads	220.573731	227.935415	216.217914
$N = M = K = 8192$, 64 threads	377.716926	406.877732	388.592315

  작은 행렬과 많은 스레드 수로 설정하여 테스트하면 `dynamic`과 `guided`의 오버헤드로 인해 `static`에서 가장 높은 성능을 보였다. 그러나, 그 외의 경우에서는 세 방식이 큰 차이를 보이지는 않았다. 행렬의 크기가 클 때는 `static`보다 `dynamic`과 `guided`에서 근소하게 높은 성능을 기록하는 결과를 볼 수는 있었다.

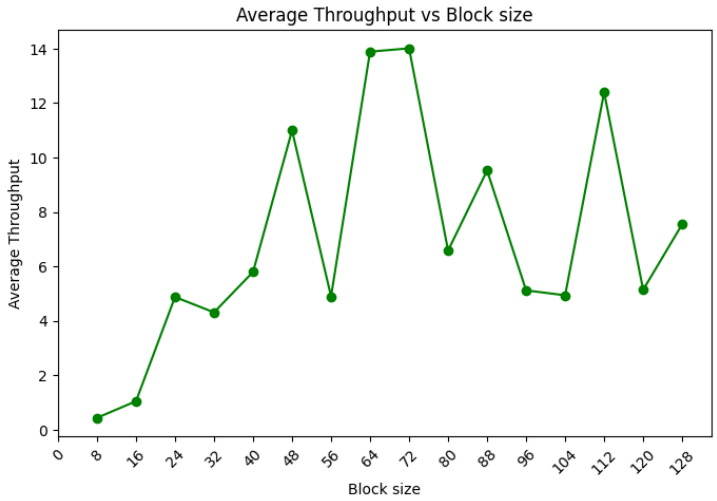
2 Estimating Cache Size

1번 문제를 풀 때 사용한 코드에서는 행렬을 캐시에 맞는 크기로 나누어 연산하는 캐시 블로킹 방식을 이용하였다. 이 문제에서는 캐시의 크기를 추정하기 위해, 블록 크기 B와 N, M, K를 모두 같게 하여 행렬곱을 수행할 때의 working set이 행렬 A, B, C 전체가 되도록 하였다. 이때 각 행렬당 B^2, 총 $3B^2$ 개의 float에 대해 연산을 수행하므로, working set의 크기는 $12B^2$ 바이트이다.

스레드 수를 1로 고정한 채 B를 점점 늘려가며 성능을 측정하면 몇몇 구간에서 성능이 갑자기 하락하는 지점들이 나오고, 그 지점들에서 요구되는 working set의 크기가 L1/L2/L3 캐시의 크기와 같을 것이라고

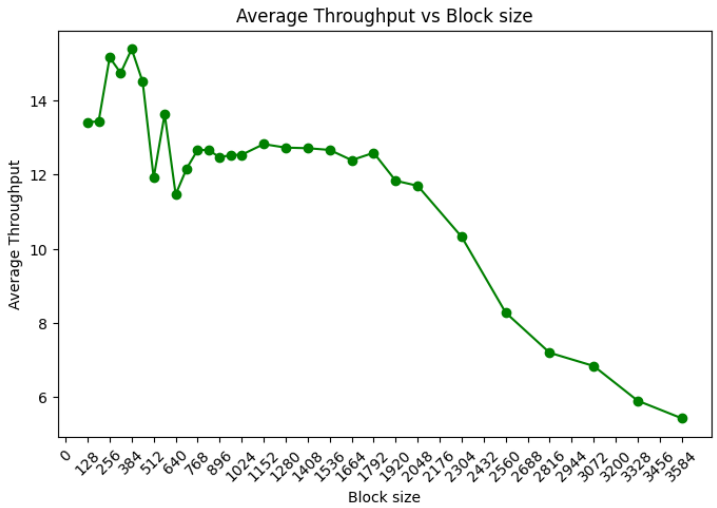
판단할 수 있다. 또한, 스레드 수를 다르게 하여 측정했을 때 하락 구간이 다르게 나오는 부분이 있다면, 그 부분과 연관된 캐시는 shared cache라고 판단할 수 있다.

- **L1 cache의 크기** 먼저, 행렬의 크기가 작은 경우에 대해 성능을 측정하여 L1 캐시의 크기를 추정하였다. B를 8부터 128까지 8씩 늘려가며 각각 3회씩 반복하여 평균 성능을 측정하였고, 그 결과는 아래와 같다.



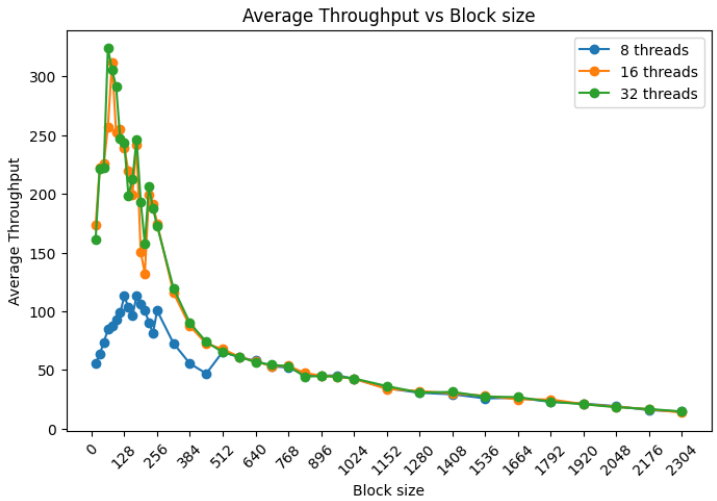
  다소 성능의 편차가 심했지만, 전반적으로는 $B \leq 72$ 구간에서는 성능이 증가했다가 그 이후로 하락하는 모습을 보였다. 성능이 하락하기 직전 블록 크기인 72를 기준으로 L1 캐시의 크기를 추정해보면, $12 \cdot B^2$ Byte $= 12 \cdot 72^2$ Byte $= 60.75$ KiB로 추정할 수 있었다.

- **L2/L3 cache의 크기** 이제 좀 더 큰 행렬에 대한 성능을 측정하는 실험을 수행하였다. B를 128부터 3584까지 늘려가며 성능을 측정하였고, 그 결과는 아래와 같다.



  $B = 320$ 전후로 성능이 상승하다가 하락하는 구간이 나타났으며, 또 $B = 1792$ 전후로는 성능이 유지되다가 하락하는 구간이 나타났다. 이를 토대로 L2 캐시의 크기는 $12 \cdot B^2$ Byte $= 12 \cdot 320^2$ Byte $\approx$ 1.17 MiB로, L3 캐시의 크기는 $12 \cdot B^2$ Byte $= 12 \cdot 1792^2$ Byte $= 36.75$ MiB로 추정할 수 있었다.

- **private 또는 shared cache 확인** private cache는 각 코어가 자신만의 캐시를 가지지만, shared cache는 모든 코어가 하나의 캐시를 공유한다. 따라서, 다시 $B < N = M = K = 4096$로 두어 캐시 블로킹이 작동하도록 하고, B를 16부터 2304까지 늘려가며 성능을 측정하였다. 스레드 수가 8, 16, 32일 때 각각 측정한 결과를 비교하였고, 그 결과는 아래와 같다.



$B < 512$일 때는 스레드 수가 다르더라도 성능의 전반적인 상승과 하락이 비슷한 구간에서 바뀌는 것을 확인할 수 있다. 따라서, L1 캐시와 L2 캐시로 인해 나타나는 성능 변화 구간은 스레드 수에 따라 크게 바뀌지 않는다고 추측할 수 있다. 또한, $B > 512$일 때도 스레드 수와 관계없이 성능이 전반적으로 하락함을 확인할 수 있지만, 앞에서 $B = 1792$ 전후로 성능이 유지되다가 하락한 것과는 달리 성능이 꾸준히 하락함을 알 수 있었다. 이는 L3 캐시에 대한 성능 변화 구간이 앞당겨졌음을 의미한다고 해석해볼 수 있다. 즉, L3 캐시는 스레드 수에 영향을 받는다고 추측할 수 있다. 따라서 L1 캐시와 L2 캐시는 private cache, L3 캐시는 shared cache로 추정해볼 수 있다.

- **추정값과 실제값 비교** 인터넷 검색 결과, 실제 Intel Xeon Silver 4216 CPU의 캐시 크기는 L1 cache 32 KiB(private), L2 cache 1 MiB(private), L3 cache 22MiB(shared)로 확인되었다. 전반적으로 L2 캐시의 경우 추정값이 실제와 유사하였으나, L1 캐시와 L3 캐시의 크기는 실제보다 더 크게 추정되었다. L1 캐시의 경우는 실험 환경에서 오는 결과의 편차가 영향을 미쳤을 가능성이 있고, L3 캐시의 경우는 계산 노드가 실제로는 2개의 소켓을 가지고 있다는 점이나 스레드 스케줄링 등이 영향을 미쳤을 수 있다.